

Jadoo AR/VR Real Estate Platform

Complete Project Documentation

Repository: **Yash-Padme/Jadoo--AR-VR-Real-Estate-Platform**

Generated on: **September 27, 2026**

This document is derived from source files and workflow definitions present in the repository.

Table of Contents

Jadoo AR/VR Real Estate Platform — Technical Documentation	
Title Page	
Jadoo AR/VR Real Estate Platform	
Executive Summary	
Problem Statement	
Target Users	
Product Goals	
Feature Inventory	
High-Level Architecture	
Repository Structure	
Frontend Walkthrough	
Entry and Providers	
Router and Layout	
Authentication Screens	
Tenant View	
Owner Property Form	
Backend Walkthrough	
Bootstrapping	
Express App and Middleware	
Auth Middleware	
Authentication Flow	
Property Management Flow	
Data Models	
`User` (`server/src/models/user.model.js`)	
`Property` (`server/src/models/property.model.js`)	
`Feedback` (`server/src/models/feedback.model.js`)	
API Endpoint Reference	
Base Prefixes	
User Routes (`server/src/routes/user.routes.js`)	
Property Routes (`server/src/routes/property.routes.js`)	
Frontend Routes and User Journeys	
Routes (`client/src/App.js`)	
Journey: New user to tenant browsing	
Journey: Owner listing creation	
AR/VR/Panorama Implementation (Actual)	
Maps, Payment, Chatbot, Text-to-Speech, Logout Components	
Maps / Location	
Payment	
Chatbot	
Text-to-Speech	
Logout	
Configuration and Environment Variables	
Backend `.env` variables inferred from code	

Frontend runtime behavior notes	
Local Development Setup	
Prerequisites	
Backend	
or npm start	
Frontend (Create React App)	
Build, Test, and Deployment Instructions	
Frontend scripts (`client/package.json`)	
Backend scripts (`server/package.json`)	
Tests	
CI/CD Explanation and Current Workflow Reality	
Security Considerations	
Validation and Error Handling	
Backend	
Frontend	
Behavioral caveat	
Known Limitations and README Discrepancies	
Roadmap and Future Improvements	
Troubleshooting Guide	
Login seems successful but protected routes fail	
Property upload fails	
Tenant page returns no properties	
Maps not loading	
CI deploy workflow failing	
Interview Walkthrough	
60-second pitch	
5-minute demo script	
Architecture explanation talking points	
Key design decisions	
Likely interviewer questions and strong answers	
Trade-offs and limitations	
Suggested improvement discussion	
Regenerating This PDF	
Source file	
Generation command	
Output file	

Jadoo AR/VR Real Estate Platform — Technical Documentation

Repository: `Yash-Padme/Jadoo--AR-VR-Real-Estate-Platform`

Documentation Type: Implementation-grounded project dossier

Generated on: September 27, 2026

Jadoo AR/VR Real Estate Platform

Comprehensive Engineering, Architecture, and Interview Walkthrough Documentation

Prepared from the repository source code and workflow configuration currently available in this project checkout. Claims in this document are based on implemented code paths unless explicitly marked as README aspiration or roadmap.

1. [Executive Summary](#executive-summary)
2. [Problem Statement](#problem-statement)
3. [Target Users](#target-users)
4. [Product Goals](#product-goals)
5. [Feature Inventory](#feature-inventory)
6. [High-Level Architecture](#high-level-architecture)
7. [Repository Structure](#repository-structure)
8. [Frontend Walkthrough](#frontend-walkthrough)
9. [Backend Walkthrough](#backend-walkthrough)
10. [Authentication Flow](#authentication-flow)
11. [Property Management Flow](#property-management-flow)
12. [Data Models](#data-models)
13. [API Endpoint Reference](#api-endpoint-reference)
14. [Frontend Routes and User Journeys](#frontend-routes-and-user-journeys)
15. [AR/VR/Panorama Implementation (Actual)](#arvrpanorama-implementation-actual)
16. [Maps, Payment, Chatbot, Text-to-Speech, Logout Components](#maps-payment-chatbot-text-to-speech-logout-components)
17. [Configuration and Environment Variables](#configuration-and-environment-variables)
18. [Local Development Setup](#local-development-setup)
19. [Build, Test, and Deployment Instructions](#build-test-and-deployment-instructions)
20. [CI/CD Explanation and Current Workflow Reality](#cicd-explanation-and-current-workflow-reality)
21. [Security Considerations](#security-considerations)
22. [Validation and Error Handling](#validation-and-error-handling)
23. [Known Limitations and README Discrepancies](#known-limitations-and-readme-discrepancies)

- 24. [Roadmap and Future Improvements](#roadmap-and-future-improvements)
- 25. [Troubleshooting Guide](#troubleshooting-guide)
- 26. [Interview Walkthrough](#interview-walkthrough)
- 27. [Regenerating This PDF](#regenerating-this-pdf)

Executive Summary

Jadoo is a full-stack JavaScript real-estate web application with a React frontend (`client/`) and an Express + MongoDB backend (`server/`). The implemented experience supports:

- user registration/login,
- token-backed authenticated API access,
- property creation with image upload,
- property listing and search,
- panorama viewing using Panolens/Three.js,
- auxiliary UI components for maps, text-to-speech, FAQ-like chatbot, and a payment demo screen.

The current source does **not** implement true browser AR overlays, WebXR scene logic, or A-Frame components despite README marketing text.

Problem Statement

Real-estate discovery is traditionally limited to static media and fragmented communication. Jadoo attempts to improve remote property exploration by combining listing management with richer visual interactions (panorama view) and onboarding flows for both owners and tenants.

Target Users

- **Property owners / sellers:** create listings with details and one image (`client/src/Owner/PropertyForm.js`).
- **Tenants / buyers:** browse and search listings and view panorama/map overlays (`client/src/Tenant/Tenant.jsx`, `Tenantcard.jsx`).
- **General visitors:** browse home pages and get-started auth page (`client/src/pages/Home.jsx`, `Getstarted.jsx`).

Product Goals

1. Provide basic authentication and session state.
2. Enable CRUD operations for property records through backend APIs.
3. Support media-backed listing display with immersive-feeling panorama rendering.
4. Expose utility experiences (map preview, payment UI, chatbot prompt flow, speech playback).

Feature Inventory

Area	Implemented in source	Key files / symbols
---	---	---
Routing shell	Yes	`client/src/App.js` routes via `BrowserRouter`, `Routes`, `Route`
Auth context	Yes	`client/src/contexts/AuthContext.jsx` → `AuthProvider`, `useAuth`, `storeToken`
User registration/login	Yes	`client/src/pages/Getstarted.jsx`; backend `registerUser`, `login`
Authenticated current-user fetch	Yes	`AuthContext.userAuthentication()` + `GET /api/v1/users`
Property add/list/read/update/delete APIs	Yes	`server/src/routes/property.routes.js`; controller
Property upload to Cloudinary	Yes	`uploadOnCloudinary` in `server/src/utils/cloudinary.js`
Panorama viewer	Yes (image panorama)	`client/src/PanoramaViewer/Panorama.js` using `PANORAMA`
Dynamic map by geocoding	Yes	`client/src/components/Map.jsx`
Static map variants	Yes	`Map1.jsx`, `Map2.jsx`
Text-to-speech	Yes	`client/src/components/TextToSpeech.jsx` (`window.speechSynthesis`)
Voice-input FAQ chatbot UI	Yes (rule-based)	`client/src/components/OptionChatbot.jsx`
Payment page	Yes (Razorpay client-side demo)	`client/src/components/Payment.jsx`
Logout route	Yes	`/Logout` + `client/src/components/LogOut.jsx`
Google Generative AI integration	**No active usage**	dependency exists in `server/package.json`

High-Level Architecture

[React SPA (CRA, React 18)]

- ■ AuthContext stores token in localStorage
- ■ Fetches backend endpoints over HTTPS URL hardcoded in components
- ■ Renders listings, map modal, panorama modal, TTS, chatbot, payment UI
- ■ Routes in client/src/App.js

[Express API (Node + MongoDB)]

- ■ app.js mounts:
 - /api/v1/users
 - /api/v1/property
- ■ JWT middleware verifyJWT checks cookie or Authorization header
- ■ Mongoose models User/Property/Feedback
- ■ Multer receives VRImage file in addProperty
- ■ Cloudinary utility uploads and cleans temporary file

Repository Structure

```
Jadoo--AR-VR-Real-Estate-Platform/
■ ■ ■ .github/workflows/main.yml
■ ■ ■ README.md
■ ■ ■ client/
■ ■ ■ ■ package.json
■ ■ ■ ■ src/
■ ■ ■ ■ ■ App.js
■ ■ ■ ■ ■ contexts/AuthContext.jsx
■ ■ ■ ■ ■ Owner/PropertyForm.js
■ ■ ■ ■ ■ PanoramaViewer/Panorama.js
■ ■ ■ ■ ■ Tenant/Tenant.jsx
■ ■ ■ ■ ■ components/{Map,Map1,Map2,Payment,OptionChatbot,TextToSpeech,LogOut}.jsx
```

```

■      ■■■ pages/{Home,Getstarted}.jsx
■■■ server/
■      ■■■ package.json
■      ■■■ src/
■          ■■■ index.js
■          ■■■ app.js
■          ■■■ db/index.js
■          ■■■ routes/{user.routes.js,property.routes.js}
■          ■■■ controllers/{user.controller.js,property.controller.js}
■          ■■■ middlewares/{auth.middleware.js,multer.middleware.js}
■          ■■■ models/{user.model.js,property.model.js,feedback.model.js}
■          ■■■ utils/{cloudinary.js,ApiError.js,ApiResponse.js,asyncHandler.js}
■■■ docs/

```

Frontend Walkthrough

Entry and Providers

- `client/src/index.js` wraps `` with `AuthProvider` and `ToastContainer`.
- `AuthProvider` tracks `token`, `user`, and `loading` in `client/src/contexts/AuthContext.jsx`.

Router and Layout

- `client/src/App.js` always renders `Navbar` and `Footer`, with route-specific page content between them.
- Routes include `/`, `/Tenant`, `/PropertyForm`, `/Getstarted`, `/Pano`, `/Logout`, `/Maps`, `/Location`, `/Payment`, `/Map1`, `/Map2`.

Authentication Screens

- `client/src/pages/Getstarted.jsx` toggles sign-up/sign-in forms.
- On success it calls `storeTokenInLS(res.data.accessToken)` and navigates to `/`.

Tenant View

- `client/src/Tenant/Tenant.jsx` fetches `/api/v1/property/getAllProperties` with `Authorization` header from context token.
- Search filters by name/title/size/price/location/area client-side.
- Each dynamic property is rendered via `Tenantcard`; two static cards (`Tenantcard1`, `Tenantcard2`) are appended.

Owner Property Form

- `client/src/Owner/PropertyForm.js` collects text fields + `VRImage` file into `FormData`.
- Sends `POST /api/v1/property/addProperty` with auth header.

Backend Walkthrough

Bootstrapping

- `server/src/index.js` loads env, imports `connectDB`, then starts Express `app` from `server/src/app.js`.
- `server/src/db/index.js` connects Mongoose using `process.env.MONGODB_URI`.

Express App and Middleware

- `server/src/app.js` configures JSON/body parser/static/cookie-parser/CORS.
- CORS allows comma-separated `CORS_ORIGIN` values with credentialed requests.
- API mounts:
- `/api/v1/users` → `user.routes.js`
- `/api/v1/property` → `property.routes.js`

Auth Middleware

- `verifyJWT` (`server/src/middlewares/auth.middleware.js`) resolves token from:
 - 1) `req.cookies.accessToken`, or
 - 2) `Authorization` header after removing any token scheme prefix.
- It loads the user, strips password/refreshToken, and assigns `req.user`.

Authentication Flow

1. **Register** (`POST /api/v1/users/register`) validates fields/email/password length.
2. User is created in MongoDB; controller generates access + refresh tokens.
3. Tokens are set as `httpOnly`, `secure` cookies and returned in JSON response.
4. **Login** (`POST /api/v1/users/login`) validates email/password and issues new tokens.
5. Frontend stores `accessToken` in `localStorage` via `AuthProvider.storeTokenInLS`.
6. On app load/token changes, frontend calls `GET /api/v1/users/user` with `Authorization:` .
7. **Logout** route `/Logout` triggers `AuthProvider.logout()` (local token removal).
8. Backend logout endpoint (`POST /api/v1/users/logout`) exists but is not called by `Logout.jsx`.

Property Management Flow

1. Authenticated user opens `/PropertyForm`.
2. Form posts `multipart/form-data` including `VRImage`.
3. Backend `upload.fields({ name: "VRImage", maxCount: 1 })` receives file.
4. `addProperty` uploads to Cloudinary, then persists property with `owner: req.user._id`.
5. Tenant page calls `getAllProperty` and displays returned records.
6. `Tenantcard` supports map popup, panorama popup, text-to-speech, and buy-button redirect.

Data Models

``User`` (`server/src/models/user.model.js``)

- Fields: ``userName``, ``email``, ``password``, ``refreshToken``.
- Hooks/methods:
- ``pre("save")`` bcrypt hash password.
- ``isPasswordCorrect(password)``
- ``generateAccessToken()``
- ``generateRefreshToken()``

``Property`` (`server/src/models/property.model.js``)

- Core fields: ``name``, ``title``, ``description``, ``location``, ``pincode``, ``price``, ``size``, ``VRImage``, ``type``, ``area``, ``amenities``, ``contact``, ``owner``.
- ``owner`` references ``User``.

``Feedback`` (`server/src/models/feedback.model.js``)

- Exists as schema/model (``Feedback``) but no connected route/controller logic currently uses it.

API Endpoint Reference

Base Prefixes

- User APIs: ``/api/v1/users`` (from ``server/src/app.js``)
- Property APIs: ``/api/v1/property`` (from ``server/src/app.js``)

User Routes (`server/src/routes/user.routes.js``)

Method	Path	Auth	Request fields	Upload	Success behavior	Error behavior
---	---	---	---	---	---	---
POST	<code>`/register`</code>	No	<code>`email`</code> , <code>`userName`</code> , <code>`password`</code> JSON	None	Creates user; sets <code>`accessToken`</code>	
POST	<code>`/login`</code>	No	<code>`email`</code> , <code>`password`</code> JSON	None	Validates credentials, sets cookies, re	
POST	<code>`/logout`</code>	Yes (<code>`verifyJWT`</code>)	none	None	Unsets refresh token in DB, clears auth co	
GET	<code>`/user`</code>	Yes (<code>`verifyJWT`</code>)	none	None	Returns authenticated user payload	401 for a

Property Routes (`server/src/routes/property.routes.js``)

All property routes are protected because ``router.use(verifyJWT)`` is applied.

Method	Path	Auth	Request fields	Upload	Success behavior	Error behavior
---	---	---	---	---	---	---
POST	<code>`/addProperty`</code>	Yes	<code>`name`</code> , <code>`title`</code> , <code>`description`</code> , <code>`location`</code> , <code>`price`</code> , <code>`area`</code> , <code>`amenities`</code>			
GET	<code>`/getAllProperties`</code>	Yes	none	None	Returns all properties via <code>`Property.find()`</code>	401 for a

GET	/getProperty/:id	Yes	path param `id`	None	Returns single property	404 if not found
PUT	/update/:id	Yes	JSON body (partial fields)	None	Updates and returns property	404 if not found
DELETE	/delete/:id	Yes	path param `id`	None	Deletes property	404 if property missing

Frontend Routes and User Journeys

Routes (`client/src/App.js`)

Route	Component	Typical journey
/	Home	Landing page with hero/sections
/Tenant	Tenant	Browse/search listings, open map/panorama, go to payment
/PropertyForm	PropertyForm	Owner adds new property
/Getstarted	Getstarted	Sign up/sign in
/Pano	PanoramaViewer	Standalone panorama route (requires VRImage prop to be meaningful)
/Logout	LogOut	Clears local token and redirects home
/Maps	DynamicGoogleMap	Raw map component route
/Location	LocationForm	Geocode form
/Payment	Payment	Razorpay demo payment page
/Map1	DynamicGoogleMap1	Static map iframe variant
/Map2	DynamicGoogleMap2	Static map iframe variant

Journey: New user to tenant browsing

1. Visit `/Getstarted` and register/login.
2. Token stored in localStorage via `AuthProvider`.
3. Visit `/Tenant`; properties fetched from backend.
4. Open card actions: View map, View AR (panorama), Buy.

Journey: Owner listing creation

1. Login, navigate `/PropertyForm`.
2. Fill details + choose image.
3. Submit to `/api/v1/property/addProperty`.
4. On success toast + redirect to home.

AR/VR/Panorama Implementation (Actual)

Implemented immersive functionality is **panorama rendering**, not full WebXR AR stack:

- `client/src/PanoramaViewer/Panorama.js` creates `PANOLENS.ImagePanorama(prop.VRImage)`.
- A `PANOLENS.Viewer` is mounted inside a referenced DOM container.

- Tenant cards open this viewer in a fullscreen-like modal.

Not found in source:

- A-Frame scene files,
- WebXR APIs (`navigator.xr`, XR session setup),
- marker tracking or real-world AR overlays.

Maps, Payment, Chatbot, Text-to-Speech, Logout Components

Maps / Location

- `Map.jsx` geocodes address+pincode using Geoapify then injects Google Maps iframe.
- `Map1.jsx` and `Map2.jsx` are fixed-coordinate iframe variants.
- `LocationForm.jsx` provides manual geocode utility inputs.

Payment

- `Payment.jsx` uses `window.Razorpay` options and opens checkout client-side.
- Contains embedded key identifiers and static prefill values.

Chatbot

- `OptionChatbot.jsx` is a UI chatbot with predefined option/response mapping and speech recognition input using `window.SpeechRecognition` fallback.
- No backend LLM/chat endpoint is wired.

Text-to-Speech

- `TextToSpeech.jsx` uses `window.speechSynthesis` to read card descriptions.

Logout

- `LogOut.jsx` calls `useAuth().logout()` to clear localStorage token and redirects to `/`.

Configuration and Environment Variables

Backend `.env` variables inferred from code

Variable	Purpose	Referenced in
---	---	---
`PORT`	Express listen port (default 8000)	`server/src/app.js`
`CORS_ORIGIN`	Allowed origins list (comma-separated)	`server/src/app.js`
`MONGODB_URI`	MongoDB connection string	`server/src/db/index.js`
`ACCESS_TOKEN_SECRET`	JWT access token signing key	`user.model.js`, `auth.middleware.js`

```
| `ACCESS_TOKEN_EXPIRY` | Access token duration | `user.model.js` |  
| `REFRESH_TOKEN_SECRET` | JWT refresh token signing key | `user.model.js` |  
| `REFRESH_TOKEN_EXPIRY` | Refresh token duration | `user.model.js` |  
| `CLOUDINARY_CLOUD_NAME` | Cloudinary config | `server/src/utils/cloudinary.js` |  
| `CLOUDINARY_API_KEY` | Cloudinary config | `server/src/utils/cloudinary.js` |  
| `CLOUDINARY_API_SECRET` | Cloudinary config | `server/src/utils/cloudinary.js` |
```

Frontend runtime behavior notes

- API calls in several components are hardcoded to Render URL with `|| localhost` fallback strings; due JavaScript truthiness, localhost fallback is effectively unreachable in current expressions.

Local Development Setup

Prerequisites

- Node.js (project workflow currently requests Node 16 in CI)
- npm
- MongoDB instance
- Cloudinary account credentials for upload features

Backend

```
cd server  
npm install  
npm run dev  
# or npm start
```

Frontend (Create React App)

```
cd client  
npm install  
npm start
```

App runs via CRA dev server (typically `http://localhost:3000`).

Build, Test, and Deployment Instructions

Frontend scripts (`client/package.json`)

- `npm start`
- `npm build`
- `npm test`
- `npm eject`

Backend scripts (`server/package.json`)

- ``npm run start``
- ``npm run dev``

Tests

- No project test files were found matching common ``*.test.*`/`*.spec.*`` patterns in current source.

CI/CD Explanation and Current Workflow Reality

Workflow file: ``github/workflows/main.yml``.

Configured steps:

1. checkout
2. setup-node@v3 (``node-version: '16``)
3. ``npm install``
4. ``npm run build``
5. Render deploy trigger via ``curl``

Important caveat from actual workflow logs:

- Recent failed run (``Deploy to Render``, run id ``31252238253``) fails at repository root ``npm install`` because there is no root ``package.json``.
- Error: ``ENOENT ... /Jadoo--AR-VR-Real-Estate-Platform/package.json``.
- This means CI currently does not build ``client/`` or ``server/`` with proper working directories.

Also note:

- Header in deploy step appears malformed/placeholder-like in YAML and should be reviewed before production use.

Security Considerations

Observed implementation risks from inspected code:

1. ****Hardcoded sensitive values in frontend source****
 - API key-style values and payment credentials appear directly in client components (e.g., map geocoding key in ``Map.jsx``, payment key material in ``Payment.jsx``).
2. ****Token handling split between cookie and localStorage****
 - Backend sets ``httpOnly`` cookies, but frontend simultaneously stores access token in localStorage and sends via ``Authorization``, increasing token exposure surface.
3. ****`secure: true` cookies in all environments****
 - In non-HTTPS local environments, cookie behavior may break login persistence.
4. ****No role/ownership authorization checks for property mutation endpoints****

- Any authenticated user can call update/delete against any property id in current controllers.
5. ****DOM insertion via `innerHTML` in map components****
- `Map.jsx`, `Map1.jsx`, `Map2.jsx` write iframe code directly, which is generally riskier than JSX rendering.
6. ****Excessive console logging around file uploads and env references****
- Upload flow logs request body/files and env key output statements exist in `cloudinary.js`.

Validation and Error Handling

Backend

- Uses wrapper `asyncHandler` and response helpers `ApiResponse`/`ApiError`.
- Input checks for register/login/property add.
- Auth middleware returns 401 with explicit error messages.

Frontend

- API errors are mostly surfaced via `react-toastify` toasts.
- Some components log errors only to console.

Behavioral caveat

- Several fetch calls use expression pattern:

```
"https://...onrender.com/..." || "http://localhost:8000/..."
```

The second URL is never chosen, which can surprise local development.

Known Limitations and README Discrepancies

Compared with `README.md` claims:

1. ****WebXR / A-Frame / AR overlays**** are claimed, but code primarily implements Panolens panorama viewing.
2. ****Automated CI lint/build/deploy narrative**** in README does not match functioning workflow (currently failing at root npm install).
3. ****Repository structure in README**** does not accurately reflect current React/Express folder structure and filenames.
4. ****Google Generative AI dependency**** is present in backend package manifest but has no integrated chatbot/API usage in runtime source.
5. ****Payment flow**** appears demo-oriented and client-only without server-side verification.

Roadmap and Future Improvements

Practical, source-aligned next steps:

1. Split CI jobs for `client/` and `server/` with proper `working-directory`.
2. Move API/payment/map keys into secure server-side config and env-driven client injection.
3. Replace hardcoded API host strings with centralized environment-aware utility.
4. Add property ownership authorization checks for update/delete.
5. Implement refresh token rotation/expiry handling and explicit logout API invocation from frontend.
6. Add backend + frontend automated tests.
7. If AR/WebXR is a product goal, add explicit WebXR scene modules and browser capability fallback logic.

Troubleshooting Guide

Login seems successful but protected routes fail

- Verify `accessToken` exists in localStorage.
- Ensure request `Authorization` header format is acceptable to `verifyJWT` (it strips a token scheme prefix if present; plain token also works).

Property upload fails

- Confirm `VRImage` file field is included.
- Validate Cloudinary env vars are set.
- Ensure `public/temp` exists and process has write permission.

Tenant page returns no properties

- Ensure backend is reachable and authenticated token is valid.
- Check network call to `/api/v1/property/getAllProperties` for 401.

Maps not loading

- Geocoding and embed rely on external services; network restrictions or key issues can block output.

CI deploy workflow failing

- Root cause currently: root-level `npm install` without root `package.json`.
- Fix by running install/build in `client` and/or `server` directories explicitly.

Interview Walkthrough

60-second pitch

“Jadoo is a full-stack real-estate platform built with React and Node/Express where users authenticate, owners create listings with uploaded media, and tenants browse listings with enhanced visualization via

panorama views, location maps, and assistive UX like text-to-speech. The backend uses MongoDB/Mongoose with JWT auth and Cloudinary uploads. The codebase demonstrates a modular route/controller/model architecture, context-based frontend auth state, and a CI/CD pipeline that needs restructuring for monorepo-style client/server directories."

5-minute demo script

1. Open `/` and explain overall UX shell (`Navbar`, `Footer`, `Home`).
2. Go to `/Getstarted` and walk through sign-up/sign-in behavior.
3. Show auth persistence from `AuthProvider` local token state.
4. Navigate to `/PropertyForm`, fill sample fields, explain multipart upload and backend `addProperty` path.
5. Open `/Tenant`, show fetched cards and search filter.
6. Open map modal and panorama modal from a card.
7. Show `/Payment` page as a demo checkout UI.
8. Conclude with backend architecture and CI caveat.

Architecture explanation talking points

- Frontend is **Create React App + React 18**, React Router, Framer Motion, Panolens, Three.js, react-speech-recognition (package), React Toastify.
- Backend is **Node/Express + MongoDB/Mongoose**, JWT, bcrypt, Multer, Cloudinary, and `@google/generative-ai` package present.
- Auth gate centered on `verifyJWT`, property flow centered on `addProperty`/`getAllProperty`.

Key design decisions

1. Context-based auth state (`AuthProvider`) simplifies route-level conditional nav behavior.
2. Controller separation (`user.controller.js`, `property.controller.js`) keeps route files thin.
3. Cloudinary offloads image persistence from local server storage.

Likely interviewer questions and strong answers

Q: Is this true AR/WebXR?

A: Current implementation is panorama-based immersion using Panolens/Three.js. README mentions broader AR/WebXR ambitions; those are roadmap-level, not fully implemented in code.

Q: How is authorization enforced?

A: `verifyJWT` protects private routes by validating JWT and loading user from DB. However, property ownership checks for update/delete should be added for stronger authorization.

Q: What is your biggest production risk today?

A: Secrets and key-like values in client code, plus CI pipeline mismatch for multi-package repo layout.

Q: How would you improve reliability quickly?

A: Fix CI working directories, centralize API base URL config, add API integration tests, and remove client-side credential exposure.

Trade-offs and limitations

- Fast prototype experience vs strict security hardening.
- Simple token handling vs robust refresh/session lifecycle.
- Panorama approximation vs richer AR/WebXR implementation complexity.

Suggested improvement discussion

If discussing future ownership, prioritize: security hardening, CI correctness, test coverage, role/ownership authorization, and explicit feature parity between README claims and shipped code.

Regenerating This PDF

Source file

- ``docs/Jadoo-AR-VR-Real-Estate-Platform-Documentation.md``

Generation command

```
python3 -m pip install -r docs/requirements.txt && python3 docs/generate_documentation_pdf.py
```

Output file

- ``docs/Jadoo-AR-VR-Real-Estate-Platform-Documentation.pdf``

The generator script creates a title page, auto-built table of contents from markdown headings, section-formatted body text, and page numbers.